

1. TITLE OF THE LAB REPORT EXPERIMENT

Implement the KMP algorithm and LCS.

2. OBJECTIVES/AIM

Implement and compare the KMP algorithm for string matching and the Longest Common Subsequence (LCS) algorithm for finding commonalities between sequences.

3. Problem_1

Implement the KMP algorithm to find the indices where a pattern matches a given string. Repeat for multiple string-pattern pairs.

4. IMPLEMENTATION_1

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class KMPAlgorithm {

    public static int[] computeLPSArray(String pat) {
        int M = pat.length();
        int[] lps = new int[M];

        int len = 0;
        int i = 1;
        while (i < M) {
            if (pat.charAt(i) == pat.charAt(len)) {
                len++;
                lps[i] = len;
                i++;
            } else {
                if (len != 0) {
                    len = lps[len - 1];
                } else {
                    lps[i] = 0;
                    i++;
                }
            }
        }
    }
}
```

```

    }
}
return lps;
}

```

```

public static List<Integer> KMPSearch(String txt, String pat) {
    int M = pat.length();
    int N = txt.length();
    List<Integer> indices = new ArrayList<>();

    int lps[] = computeLPSArray(pat);

    int i = 0;
    int j = 0;

    while (i < N) {
        if (pat.charAt(j) == txt.charAt(i)) {
            j++;
            i++;
        }

        if (j == M) {
            indices.add(i - j);
            j = lps[j - 1];
        }

        else if (i < N && pat.charAt(j) != txt.charAt(i)) {
            if (j != 0) {
                j = lps[j - 1];
            } else {
                i = i + 1;
            }
        }
    }

    return indices;
}

```

```

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
}

```

```

System.out.print("Enter the number of test cases: ");
int T = scanner.nextInt();
scanner.nextLine();

for (int t = 0; t < T; t++) {
    System.out.print("Enter the text: ");
    String txt = scanner.nextLine();

    System.out.print("Enter the pattern: ");
    String pat = scanner.nextLine();

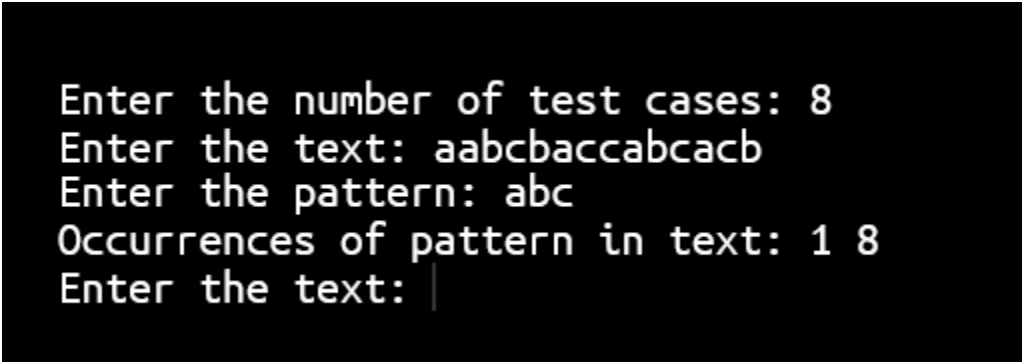
    List<Integer> indices = KMPSearch(txt, pat);

    System.out.print("Occurrences of pattern in text: ");
    for (int index : indices) {
        System.out.print(index + " ");
    }
    System.out.println();
}

scanner.close();
}
}

```

5. OUTPUT_1



```

Enter the number of test cases: 8
Enter the text: aabcbaccabcacb
Enter the pattern: abc
Occurrences of pattern in text: 1 8
Enter the text: |

```

6. Problem_2

Implement LCS and print all common subsequences in descending order of length for two given sequences.

7. IMPLEMENTATION_2

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class LCS {

    public static int[][] lcs(String X, String Y) {
        int m = X.length();
        int n = Y.length();
        int[][] L = new int[m + 1][n + 1];

        for (int i = 0; i <= m; i++) {
            for (int j = 0; j <= n; j++) {
                if (i == 0 || j == 0) {
                    L[i][j] = 0;
                } else if (X.charAt(i - 1) == Y.charAt(j - 1)) {
                    L[i][j] = L[i - 1][j - 1] + 1;
                } else {
                    L[i][j] = Math.max(L[i - 1][j], L[i][j - 1]);
                }
            }
        }
        return L;
    }

    public static void printLCS(int[][] L, String X, String Y, int i, int j, StringBuilder
currLCS, List<String> allLCS) {
        if (i == 0 || j == 0) {
            if (currLCS.length() > 0) {
                allLCS.add(currLCS.toString());
            }
            return;
        }

        if (X.charAt(i - 1) == Y.charAt(j - 1)) {
            currLCS.append(X.charAt(i - 1));
            printLCS(L, X, Y, i - 1, j - 1, currLCS, allLCS);
            currLCS.deleteCharAt(currLCS.length() - 1);
        } else {
```

```

        if (L[i - 1][j] > L[i][j - 1]) {
            printLCS(L, X, Y, i - 1, j, currLCS, allLCS);
        } else if (L[i - 1][j] < L[i][j - 1]) {
            printLCS(L, X, Y, i, j - 1, currLCS, allLCS);
        } else {
            printLCS(L, X, Y, i - 1, j, currLCS, allLCS);
            printLCS(L, X, Y, i, j - 1, currLCS, allLCS);
        }
    }
}

public static void main(String[] args) {
    String X = "AGGTAB";
    String Y = "GXTXAYB";

    int[][] L = lcs(X, Y);
    List<String> allLCS = new ArrayList<>();
    StringBuilder currLCS = new StringBuilder();

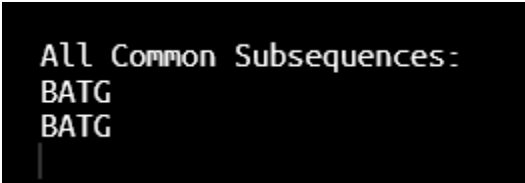
    printLCS(L, X, Y, X.length(), Y.length(), currLCS, allLCS);

    Collections.sort(allLCS, (a, b) -> b.length() - a.length());

    System.out.println("All Common Subsequences:");
    for (String lcs : allLCS) {
        System.out.println(lcs);
    }
}

```

8. OUTPUT_2



```

All Common Subsequences:
BATG
BATG

```

9. SUMMARY:

This project aims to implement and compare two fundamental string algorithms:

1. Knuth-Morris-Pratt (KMP) Algorithm:

- Efficiently finds occurrences of a pattern within a text string.
- Utilizes a precomputed table to avoid redundant comparisons.

2. Longest Common Subsequence (LCS) Algorithm:

- Finds the longest subsequence common to two given sequences.
- Employs dynamic programming for efficient computation.

The project will involve:

- Implementing both algorithms in a suitable programming language.
- Analyzing their time and space complexities.
- Comparing their performance on different input datasets.
- Exploring potential applications of these algorithms in real-world scenarios.